Python Functions Practice Set - 5

1. Parking Lot Management System (With Answer)

Question

A parking lot stores vehicle details as a list of dictionaries.

Create a function parking_report(vehicles) that returns:

- Total number of vehicles
- Number of Cars, Bikes, and Trucks
- Total parking fee collected
- Vehicle with the highest parking fee
- Average parking fee

Example Input

```
vehicles = [
  {"number":"JK02A1234","type":"Car","fee":120},
  {"number":"JK02B2345","type":"Bike","fee":50},
  {"number":"JK02C4567","type":"Truck","fee":300},
  {"number":"JK02D9876","type":"Car","fee":150}
]
```

Answer

```
def parking_report(vehicles):

  vehicle_count = {}
  total_fee = 0
  highest = vehicles[0]

  for vehicle in vehicles:

    vehicle_type = vehicle["type"]
```

```
        vehicle_count[vehicle_type] = vehicle_count.get(vehicle_type, 0) + 1

        total_fee += vehicle["fee"]

        if vehicle["fee"] > highest["fee"]:
            highest = vehicle

    average = total_fee / len(vehicles)

    return {
        "Total Vehicles": len(vehicles),
        "Vehicle Count": vehicle_count,
        "Total Fee": total_fee,
        "Highest Fee Vehicle": highest["number"],
        "Average Fee": average
    }


vehicles = [
    {"number":"JK02A1234","type":"Car","fee":120},
    {"number":"JK02B2345","type":"Bike","fee":50},
    {"number":"JK02C4567","type":"Truck","fee":300},
    {"number":"JK02D9876","type":"Car","fee":150}
]

print(parking_report(vehicles))
```

2. Restaurant Order Analytics (With Answer)

Question

A restaurant stores customer orders.

Create a function restaurant_report(orders) that returns:

- Total revenue
- Most ordered food item
- Number of unique food items
- Average order value

- List of food items costing more than the average order value

Example Input

```
orders = [
  {"item":"Burger","price":180},
  {"item":"Pizza","price":350},
  {"item":"Burger","price":180},
  {"item":"Pasta","price":250},
  {"item":"Pizza","price":350}
]
```

Answer

```
def restaurant_report(orders):

  revenue = 0
  frequency = {}

  for order in orders:

    revenue += order["price"]

    item = order["item"]
    frequency[item] = frequency.get(item, 0) + 1

  average = revenue / len(orders)

  expensive = []

  for order in orders:
    if order["price"] > average and order["item"] not in expensive:
      expensive.append(order["item"])

  popular = max(frequency, key=frequency.get)

  return {
    "Revenue": revenue,
    "Most Ordered": popular,
    "Unique Items": len(frequency),
    "Average Order": average,
    "Above Average Items": expensive
  }
```

```
orders = [
  {"item":"Burger","price":180},
  {"item":"Pizza","price":350},
  {"item":"Burger","price":180},
  {"item":"Pasta","price":250},
  {"item":"Pizza","price":350}
]

print(restaurant_report(orders))
```

---

3. Online Exam Analytics

Question

An online exam stores every student's answers.

Each student has attempted 10 questions.

Create a function exam_report(results) that returns:

- Highest score
- Lowest score
- Average score
- Students scoring at least 8 marks
- Number of students who failed (score < 4)

Example Input

```
results = {
  "Aman":[1,1,1,1,1,1,0,1,0,1],
  "Riya":[1,1,1,1,1,1,1,1,1,1],
  "Rahul":[1,0,0,1,0,0,1,0,0,1],
  "Priya":[1,1,1,1,0,1,1,1,1,0]
}
```

Answer

```
def exam_report(results):
```

```
    scores = {}

    for student, answers in results.items():
        scores[student] = sum(answers)

    highest = max(scores, key=scores.get)
    lowest = min(scores, key=scores.get)

    average = sum(scores.values()) / len(scores)

    excellent = []

    failed = 0

    for student, score in scores.items():

        if score >= 8:
            excellent.append(student)

        if score < 4:
            failed += 1

    return {
        "Highest": highest,
        "Lowest": lowest,
        "Average": average,
        "Excellent Students": excellent,
        "Failed Students": failed
    }


results = {
    "Aman":[1,1,1,1,1,1,0,1,0,1],
    "Riya":[1,1,1,1,1,1,1,1,1,1],
    "Rahul":[1,0,0,1,0,0,1,0,0,1],
    "Priya":[1,1,1,1,0,1,1,1,1,0]
}

print(exam_report(results))
```

Without answers

1. Food Delivery Analytics System

Question

A food delivery platform stores all orders as a list of dictionaries.

Each order contains:

- Customer Name
- Restaurant Name
- Order Amount

```
orders = [
   {"customer": "Aman", "restaurant": "Domino's", "amount": 450},
   {"customer": "Riya", "restaurant": "KFC", "amount": 320},
   {"customer": "Rahul", "restaurant": "Domino's", "amount": 600},
   {"customer": "Aman", "restaurant": "Burger King", "amount": 280},
   {"customer": "Priya", "restaurant": "KFC", "amount": 520}
]
```

Create a function delivery_report(orders) that returns:

- Total revenue generated
- Restaurant with the highest number of orders
- Customer who spent the most money
- Average order amount
- List of customers who placed more than one order

2. Coding Contest Leaderboard

Question

A coding contest stores participant details as a list of dictionaries.

Each participant contains:

- Name

- Problems Solved
- Penalty Time

```
participants = [
  {"name": "Aman", "solved": 5, "penalty": 420},
  {"name": "Riya", "solved": 7, "penalty": 510},
  {"name": "Rahul", "solved": 7, "penalty": 480},
  {"name": "Priya", "solved": 4, "penalty": 300},
  {"name": "Karan", "solved": 6, "penalty": 450}
]
```

Create a function contest_report(participants) that returns:

- Contest winner (highest problems solved; if tied, lowest penalty wins)
- Average number of problems solved
- List of participants who solved more than the average number of problems
- Total problems solved by all participants
- Number of participants who solved at least 5 problems

3. Cinema Ticket Booking System

Question

A cinema hall has **100 seats**.

The list below contains the seat numbers booked by customers.

bookings = [2, 4, 5, 8, 12, 14, 18, 20, 25, 28, 35, 40, 45]

Create a function cinema_report(bookings) that returns:

- Number of booked seats
- Number of vacant seats
- Occupancy percentage
- Whether the hall is **"House Full"** (100% occupied) or **"Seats Available"**
- First 10 vacant seat numbers